Design layer

the whole layer, as one document

Contents

ROOT DESIGN

Jido Composer — migration draft

1 Jido Composer — migration draft

1.1 Foundation

Jido Composer is an Elixir library for combining deterministic workflows and model-selected operations through one node contract.

GOAL

Compose deterministic and adaptive agent flows

A consumer can nest workflows and orchestrators in either direction without changing the parent's node contract.

GOAL

Carry results through nested and paused work

A consumer can combine sequential, parallel, collection, and human-input operations while retaining their context and failure information.

NO-GOAL

Replace the Jido runtime or its primitives

Composer does not replace Jido Actions, Agents, AgentServer, Signals, or Persist.

NO-GOAL

Choose host notification and timeout infrastructure

Notification transport and durable timeout scheduling remain host responsibilities.

INVARIANT

Children cannot update parent ambient context

The composition layer scopes returned results into working context. It never uses a child result to update parent ambient values.

mechanism

INVARIANT

Suspension does not consume a workflow transition

The reserved suspend outcome holds the current workflow state. A validated resume outcome or timeout outcome selects the next transition.

mechanism

PRINCIPLE

Keep composition independent of execution choice

The same configured node participates in deterministic and adaptive compositions through explicit execution contracts.

PRINCIPLE

Describe effects separately from control decisions

Strategies describe runtime work through directives. Exceptions and unresolved claims about this separation remain explicit in the pending decisions.

1.2 Pending updates

ruling **Approve the migrated design before promotion** 2026-09-02

The original Markdown remains unchanged. Approval applies to this draft's organization, extracted rationale, and explicit qualifications.

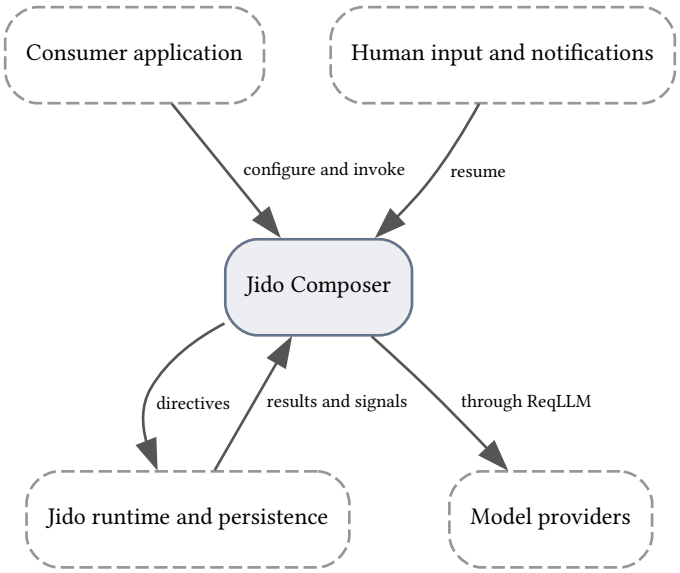
ruling **Complete implementation conformance and coverage** 2026-09-02

> The source corpus establishes carried intent. The approved sync corrections describe observed APIs and limitations, supported by 195 focused passing tests and targeted probes. Repository coverage, confirmed behavior areas, and unresolved safety decisions remain incomplete; this is not a clean conformance result.

1.3 System at a glance

The core-model context owns shared meaning and execution values. The execution context explains how consumers and runtime integrations operate on them.

ALTITUDE L1 · BOUNDARY Composer within its host

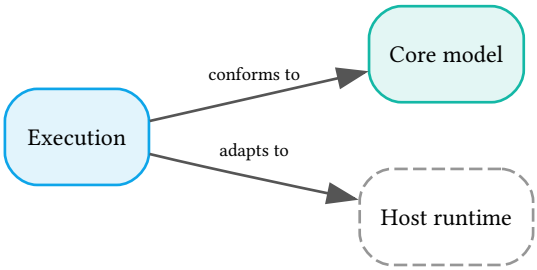


Composer owns composition rules. The host owns provider credentials, notification delivery, storage configuration, and durable timeout scheduling.

core-model
Composition definitions, result values, execution lifecycles, approval records, and their relationships.

execution
Node adapters, strategy protocols, LLM calls, checkpointing, observation, and consumer examples.

ALTITUDE L2 · CONTEXTS Two documentation contexts



These are documentation boundaries, not a proposed package reorganization. Execution adopts the core model's vocabulary and values; the host performs effects.

1.4 Ownership and cross-context contract

Logical execution owners remain the existing workflow and orchestrator strategies. The core model describes their values without moving implementation ownership.

Aggregate or value family	Consistency owner	Execution participant
Workflow run and Machine	Workflow strategy	Workflow DSL and AgentServer

Orchestration run and tool calls	Orchestrator strategy	AgentTool, LLMAction, AgentServer
Parallel execution	Parent strategy's FanOut.State	FanOutBranch executor
Approval and suspension	Suspended strategy or held tool call	HumanNode, notification host, Resume
Checkpoint and child tracking	Agent lifecycle and parent strategy	Jido.Persist and AgentServer
Skill definition	Consumer configuration	Skill.assemble and DynamicAgentNode

- Authority: the strategy accepts or refuses results and resumes before changing its execution value. The runtime executes directives but cannot independently advance the Machine.
- Correlation: workflow state and child request ID identify child work; orchestration uses tool call ID; suspension and approval each retain their own request ID. Child references preserve agent module, agent ID, and tag across process replacement.
- Mutation: boundaries exchange values or signals. A child returns a result, never a mutable reference to the parent's context.
- Failure: node errors retain their original reason. Unmatched workflow transitions fail explicitly; unknown tool and resume identifiers are refused according to the protocol's stated rules.
- Idempotency: matching pending IDs prevents stale transitions. Checkpoint compare-and-swap is a separate storage contract; it does not prove exactly-once execution of external effects.

GROUNDING

A research orchestrator can invoke an ETL workflow as one tool. The tool call ID correlates its answer, while the workflow scopes ExtractAction's records under extract and preserves its own transition history.

1.5 Reading and source coverage

Read the core model first for the shared contract, then execution for the mechanisms and complete integration details.

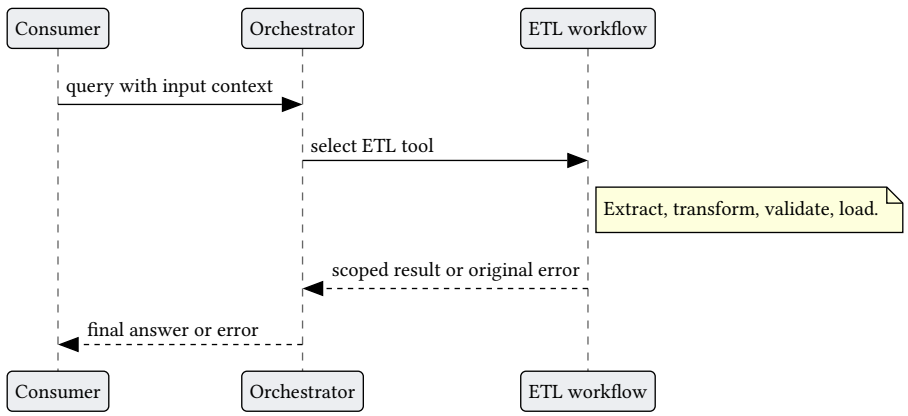
- The migration map accounts for all 30 original documents, including both limitation notes and all eight use cases. It distinguishes consolidated content from unresolved source conflicts.
- The original glossary is split into project-specific terms. General Jido and engineering concepts are explained at their integration boundary rather than duplicated as project-owned terminology.
- Existing rationale is extracted into proposed decision records. No new technology selection or model recommendation is implied by historical example identifiers.

Source-to-section migration map Documented subsystem coverage Reproducible document build

1.6 End-to-end walkthrough

A consumer composes a research coordinator with an ETL workflow and retains one contract across their boundary.

SEQUENCE One consumer request



The parent chooses a tool; the child owns its internal sequence. Direct synchronous and AgentServer execution differ in transport, not in the intended composition contract.

- AgentNode exposes the configured ETL workflow. The orchestrator correlates its invocation by call ID; the workflow scopes results by state name.
- A child suspension keeps the parent waiting. The accepted result becomes context data and a tool-result message; the final consumer result is unwrapped from NodeIO.

CONTEXT

Core model

2 Core model

2.1 Foundation

The core model describes the values shared by all compositions and the rules that transform them.

GOAL

Keep nested compositions substitutable

A parent consumes a node result without knowing the child's internal strategy.

NO-GOAL

Introduce replacement runtime entities

The census is a logical view of existing documented values. Separating configuration from execution here does not mandate splitting an Elixir struct.

INVARIANT

Result adaptation precedes scoped accumulation

The composition owner resolves a result into a map before storing it under the node's scope.

mechanism

INVARIANT

Only declared resume outcomes advance human work

Human resume checks request identity, permitted decision, and response data before applying a transition.

mechanism

PRINCIPLE

Separate definitions from execution history

Configuration describes possible work; execution values record progress; events record what was received or happened.

2.2 Pending updates

ruling **Settle the scope of the algebraic laws** 2026-09-02

The foundations assert an endomorphism monoid and associative deep merge. Context Flow permits repeated scopes and replacement of non-map values; the laws need explicit preconditions and an identity operation compatible with scoping.

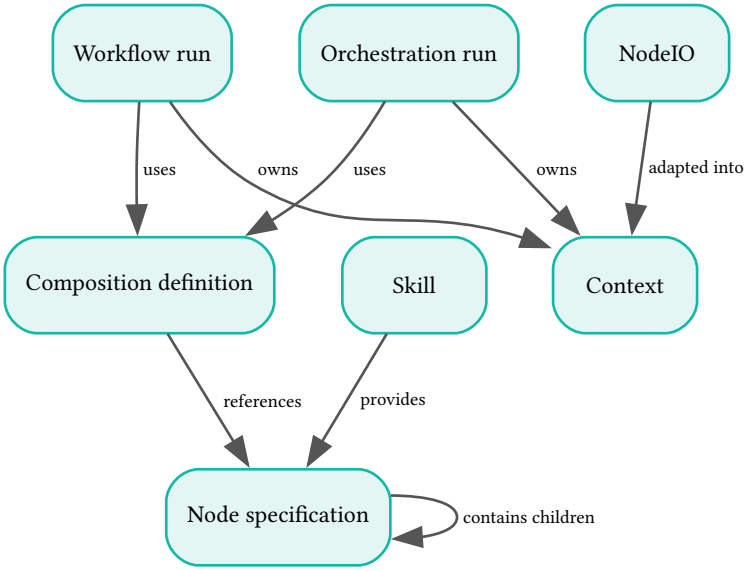
ruling **Specify approval withdrawal after dispatch** 2026-09-02

The sources define rejection before execution and sibling cancellation. They do not define revoking an already accepted approval while the approved call executes. No revocation behavior is invented here.

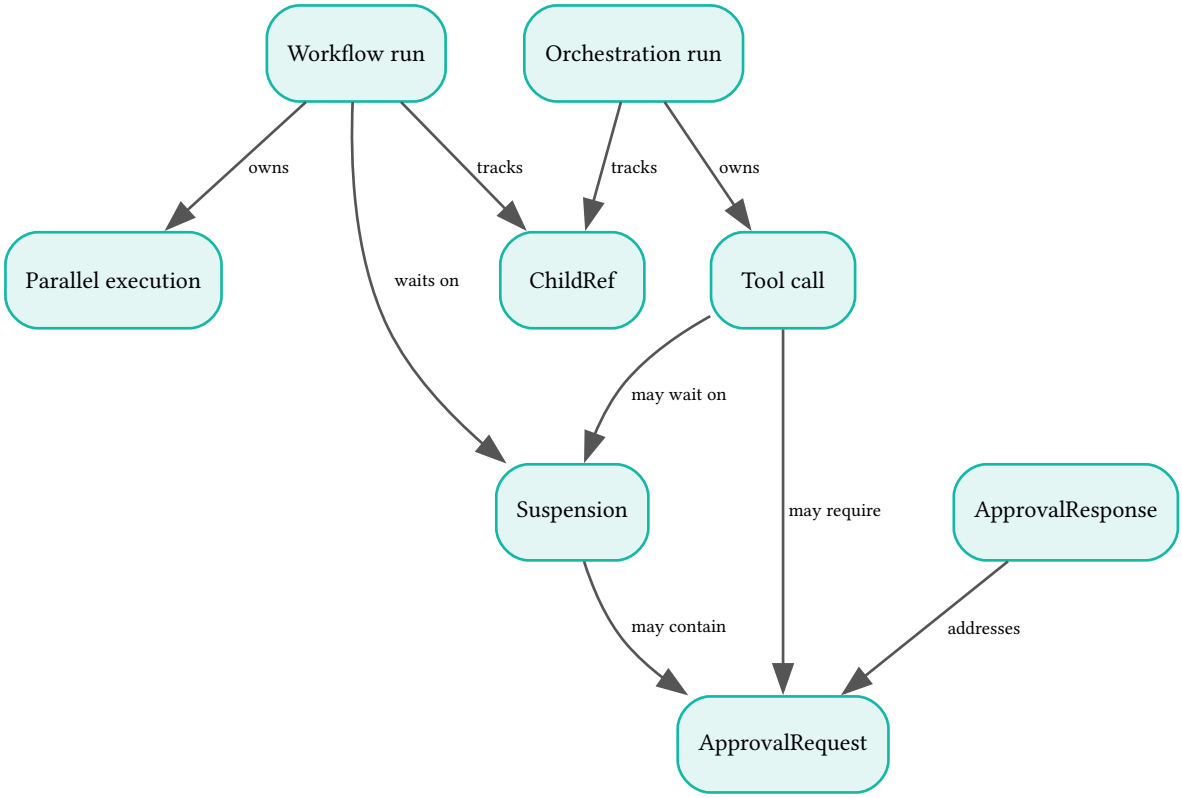
2.3 System at a glance

Composition is one domain: definitions select operations, execution state tracks them, and results supply the facts for the next decision.

ALTITUDE L4 · INTERNALS Definitions and recursive composition



The two run aggregates each use one composition definition and one Context. A definition references zero or more Node specifications; a specification can recursively contain child specifications. Skill supplies reusable operations. NodeIO is adapted before scoped accumulation.



The workflow strategy owns parallel collection progress. Both run aggregates track child references. An orchestration owns zero or more Tool calls; a held call has at most one current ApprovalRequest and a paused call at most one current Suspension. A human suspension contains one request; each ApprovalResponse addresses one request. These two views together index the complete census below.

- core-model owns these descriptions. execution supplies the operations that realize them through the existing strategy and runtime APIs.
- Configuration, progress, and observed results are distinct logical aspects. The Machine implementation may hold all three; this view assigns each fact a provenance without claiming that the implementation already separates its storage.
- A node definition may be reused in many compositions. A Skill provides zero or more tool modules, and assembly selects zero or more Skills to create one orchestrator configuration.

GROUNDING

In the ETL example, the transition from validate on invalid is authored configuration. The current state validate is computed progress; the received invalid outcome is evidence interpreted through that configuration.

2.4 Definitions and values

Definitions are supplied by the consumer. Derived definitions are created by wrapping, schema conversion, or skill assembly before execution.

Node specification

value-object immutable composition owned by Consumer configuration

The configured operation reused at a composition position.

ATTRIBUTES

authored **Name** · Node name
The configured identifier; action and agent adapters may derive it from module metadata.

authored **Description** · Human-readable operation description
The description exposed to a parent and, when supported, a model tool.

authored **Parameters** · Optional input schema
The allowed input fields and constraints declared by the operation.

authored **Configuration** · Node-kind options
Execution mode, timeouts, child selection, and other options are specified in the execution chapter's adapter contracts.

RELATIONSHIPS

`n : 0..n` A specification may contain child node specifications through parallel, traverse, or an agent composition.

Composition definition

value-object immutable composition owned by Consumer configuration

The consumer's declared rules for selecting nodes and ending a run.

ATTRIBUTES

authored **Workflow rules** · State bindings and transition map
A workflow declares its initial state, node bindings, outcome-to-state map, terminal states, and success states.

authored **Orchestrator rules** · Tool selection and generation configuration
An orchestrator declares available operations, prompt, model, limits, and optional termination action.

authored **Boundary rules** · Ambient key set and fork transformations
These determine what input is ambient and how child ambient values are derived.

RELATIONSHIPS

`1 : 0..n` A definition references configured node specifications.

Context

value-object immutable composition owned by Parent strategy

The current data value read by nodes and updated by the composition owner.

ATTRIBUTES

derived **Ambient values** · Read-only named values
Extracted at start and derived at each child boundary from input and declared fork functions.

derived **Working values** · Map of input and scoped result maps
Computed at result acceptance from the previous context and the accepted result.

authored **Fork transformations** · Named module-function-arguments tuples
Each transformation takes ambient and working values and returns child ambient values.

NodeIO

value-object immutable composition owned by Composition result adapter

The typed result value adapted by the composition layer.

ATTRIBUTES

observed **Output** · Tagged map, text, or object
The node or model supplies the value; accepting it is a separate composition decision.

authored **Object schema** · Optional JSON Schema
The schema associated with structured output, when one is provided.

Skill

value-object · immutable · composition · owned by Consumer skill registry

A reusable capability definition independent of one execution.

ATTRIBUTES

- authored **Name** · Unique skill name
  The registry selection identifier.
- authored **Description** · Capability description
  The information a caller or parent model uses to select the capability.
- authored **Prompt fragment** · Instruction text
  A self-contained fragment combined with the base prompt during assembly.

RELATIONSHIPS

- `1 : 0..n` A Skill provides action or agent modules adapted to node specifications.

ApprovalRequest

entity · immutable · composition · owned by Suspended strategy

The immutable question and response constraints of one human-input attempt.

ATTRIBUTES

- derived **Request ID** · Unique correlation identifier
  Generated when the request is constructed.
- derived **Question and visible data** · Prompt text and context subset
  Computed at request creation from the configured prompt and selected context keys.
- authored **Response constraints** · Allowed outcomes and optional data schema
  Declared on the HumanNode or approval policy.
- authored **Timeout policy** · Duration or infinity and timeout outcome
  The maximum wait and the outcome selected on expiry.
- observed **Creation time** · Instant
  The clock reading recorded when the request is created.
- derived **Flow location** · Agent identity and node or call location
  Enriched by the strategy with agent module, agent ID, node name, and workflow state or tool call.
- authored **Notification metadata** · Map of host-specific values
  Metadata for the external notification system.

RELATIONSHIPS

- `1 : 0..n` Candidate ApprovalResponses address this request; the owner accepts only a valid response while the request remains pending.

ApprovalResponse

event · immutable · composition · owned by Respondent and receiving strategy

The external response submitted for validation against one request.

ATTRIBUTES

- observed **Decision** · Outcome atom
  The submitted decision is not authority until the request owner validates it.
- observed **Data** · Optional response map
  Additional submitted values checked against the request's schema.

observed **Respondent** · Opaque respondent identity
Identity information supplied by the host; authentication is not defined by this record.

observed **Comment** · Optional text
The respondent's explanation.

observed **Response time** · Instant
The recorded response timestamp.

RELATIONSHIPS

n : 1 request_id addresses exactly one ApprovalRequest.

2.5 Execution census

Execution values track work under an agent identity. Their transitions are computed from configuration and accepted events.

Workflow run

aggregate stateful composition owned by Workflow strategy

One workflow's progress and accepted result history.

ATTRIBUTES

derived **Current state** · Declared workflow state
Computed at initialization and at every accepted transition.

derived **Transition history** · Ordered state-outcome-time entries
Appended at transition time; event timestamps come from observed clock values.

observed **Original error** · Optional failure reason
The original node, child, or transition failure retained for the caller.

RELATIONSHIPS

n : 1 Each run uses one Composition definition.

1 : 1 Each run owns one current Context value.

1 : 0..n A run may track child references and branch executions; the Machine still has only one current state.

Orchestration run

aggregate stateful composition owned by Orchestrator strategy

One model-directed execution and its conversation and call progress.

ATTRIBUTES

derived **Status** · Orchestration lifecycle state
Computed from model progress, active calls, held approvals, and suspended calls.

derived **Iteration** · Nonnegative turn count
Updated at model-turn boundaries and checked against the declared limit.

derived **Conversation** · Ordered model and tool messages
Constructed at each turn from observed responses and accepted tool results.

derived **Final result** · Optional output or original error
Selected at completion; structured successful results are wrapped as NodeIO objects.

RELATIONSHIPS

n : 1 The run uses one Composition definition's available tools and generation configuration.

1 : 1 The run owns the Context used to construct tool arguments and accumulate accepted results.

1 : 0..n The run owns Tool calls, child references, and suspensions.

Tool call

entity · stateful · composition · owned by Orchestrator strategy

One invocation selected by the model and tracked to resolution.

ATTRIBUTES

observed **Call ID** · Invocation identifier
Received with the model's request and retained for result correlation.

observed **Arguments** · Parsed parameter map
Model-supplied arguments are checked and translated at the adapter boundary.

derived **Progress** · Queued, held, executing, suspended, or resolved
Computed from dispatch, approval, and result events.

RELATIONSHIPS

n : 1 A regular call names one available node specification.

1 : 0..1 A held call has one current ApprovalRequest.

1 : 0..1 A paused call has one current Suspension.

Parallel execution

entity · stateful · composition · owned by Parent strategy FanOut.State

The parent-owned collection state for FanOutNode or MapNode work.

ATTRIBUTES

derived **Execution ID** · Unique parallel operation identifier
Generated at dispatch for branch correlation.

derived **Branch progress** · Queued, pending, completed, and suspended collections
Updated when branches are dispatched, return, or resume.

derived **Accepted results** · Named results or indexed results
Collected at result acceptance; MapNode preserves input order.

authored **Merge and failure policies** · Declared result and error policies
FanOutNode supports default scoped deep merge or a custom function, and fail-fast or partial collection.

RELATIONSHIPS

1 : 0..n A parallel execution invokes child node specifications; MapNode reuses one specification over the input collection.

Suspension

entity · stateful · composition · owned by Suspended strategy

One paused computation tracked until continuation or expiry.

ATTRIBUTES

derived **Suspension ID** · Unique pause identifier
Generated for correlation independently of any enclosing agent identity.

observed **Reason** · Human input, rate limit, async completion, external job, or custom
The node or policy supplies the pause reason.

observed **Created at** · Instant
Recorded at suspension creation.

authored **Continuation policy** · Resume signal, duration or infinity, timeout outcome
Declares the expected continuation and expiry policy.

observed **Reason metadata** · Optional map
Carries reason-specific information.

derived **Pending membership** · Active or resolved
Derived at read from the owner's pending state, not an added status field on the Suspension struct.

RELATIONSHIPS

`1 : 0..1` A human-input suspension contains one ApprovalRequest; other reasons contain none.

ChildRef

entity stateful composition owned by Parent strategy Children collection

The stable identity and lifecycle reference retained when a child process changes.

ATTRIBUTES

derived **Child identity** · Agent module and agent ID
Established at child creation or recovered from checkpoint metadata.

derived **Tag** · Parent-assigned correlation value
Derived from the parent state or tool call at dispatch.

observed **Checkpoint key** · Optional storage address
Recorded from the child's checkpoint notification.

derived **Status** · Running, paused, hibernated, completed, or failed
Updated from child lifecycle events.

derived **Phase** · Spawning, awaiting result, or absent
The phase determines whether replay re-spawns or keeps waiting.

RELATIONSHIPS

`n : 0..1` A ChildRef may identify the Suspension that caused the child to pause.

2.6 Context and result transformations

The composition owner performs scoping and adaptation. Nodes return raw result values and do not add their own scope.

Operation	Rule	Concrete result
Scope a workflow result	Use the current state name	extract returning records produces extract.records
Scope an orchestrator result	Use the tool name	Repeated research results occupy research
Merge nested maps	Merge their entries recursively	a.x and a.y both remain
Merge non-map values	Right-hand value replaces left	items [2] replaces items [1]
Flatten context	Reserve the tuple ambient key	Other top-level keys remain working data
Fork for a child	Transform ambient; copy working input	A derived child trace ID does not replace the parent's

GROUNDING

An extract result with records and count is stored as extract: (records: ..., count: 5). A downstream transform reads extract.records; a repeated extract may read its prior scope before returning an updated list.

- Ambient keys are split from the initial input. The flattened ambient marker is the tuple returned by Context.ambient_key(), not a user string or atom.
- Fork functions receive ambient and working values and return transformed ambient values. They run at agent boundaries; ordinary parallel branches share ambient without an additional fork.
- A child returns only its result. The parent scopes that value in working context, so it cannot replace parent ambient context.

2.6.1 Output adaptation

The adapter makes heterogeneous node outputs acceptable to scoped map accumulation.

Input	Map for accumulation	Consumer result
NodeIO map	Underlying map	Underlying map
NodeIO text	text: value	Text
NodeIO object	object: value	Object
Bare string	text: value	String
Other term	value: term	Original term where the public API permits

- Machine.resolve_result handles envelopes, maps, strings, and the documented fallback for other terms. FanOut resolves each branch before merging; AgentTool unwraps for model serialization.
- Input and output type declarations may warn about incompatible adjacent nodes. They do not reject a composition whose runtime adapter can resolve the value.
- AgentNode.run wraps a child query result under result. The workflow synchronous child helper returns the raw child result; Machine adaptation therefore must also accept bare text.

2.7 Composition operations and algebra

Five constructors describe fixed composition structure. Model-selected invocation adds runtime selection of the next operation.

Operation	Composition meaning	Concrete carrier
Sequence	Run the next operation after the prior outcome	Workflow transitions
Parallel	Run fixed named children and merge	FanOutNode
Choice	Choose a declared outcome edge	Transition lookup
Traverse	Apply one node to a runtime collection	MapNode
Identity	Pass the input through	Pass-through operation
Bind	Select the next invocation from runtime results	Orchestrator

- Sequence can contain parallel work; parallel branches can contain sequences; traverse can contain agents, FanOutNode, or HumanNode. The same contract recurs at each level.
- FanOutNode selects a fixed set of different branches and returns a named map. MapNode selects one child specification, determines count from input, and returns an ordered list.
- The documented category-theory interpretation treats Context as one object, nodes as endomorphisms, error-aware composition as Kleisli composition, branching as coproduct selection, and parallel execution as a product.
- The interpretation treats the orchestrator as runtime path selection in the free category generated by available nodes, streaming as an unfolding computation, and nested agents as a functorial embedding. Ambient propagation corresponds to a Reader environment; fork transformations and NodeIO adaptation correspond to natural transformations.
- Suspension is described as a partial computation that completes after continuation. These interpretations explain intent; they are not a proof that arbitrary external effects obey the algebraic laws.

Proposed law from the source	Required test or qualification
Left and right identity	Inserting a pass-through operation preserves the result under the chosen scoping convention
Associative sequential composition	Grouping pure compatible nodes does not alter outcomes or results

Error as left zero	Error short-circuits the chosen sequence
Merge associativity and identity	Specify the permitted result shapes and repeated-scope behavior
Outcome preservation	Composition preserves explicitly declared branching semantics
Nested substitution	Synchronous and directive paths preserve the same observable result contract
Read-only environment	Child execution leaves parent ambient values unchanged

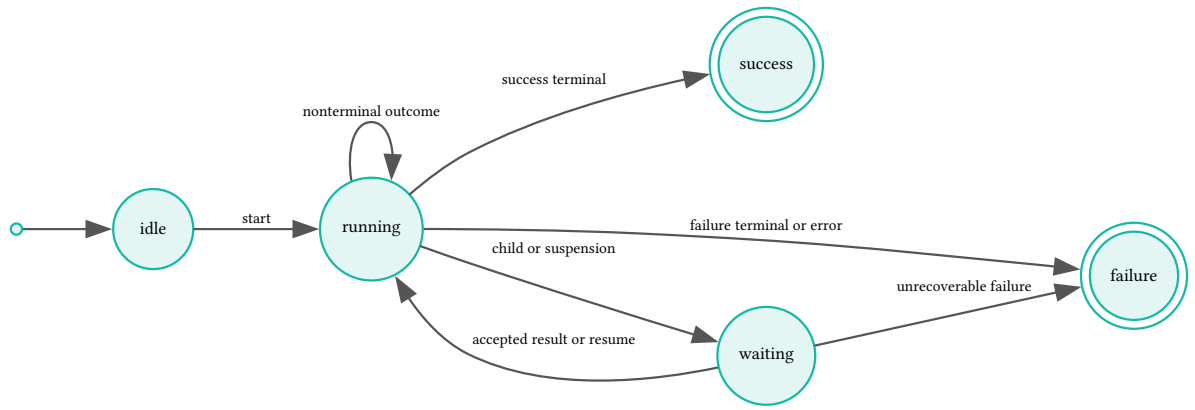
2.8 Lifecycles and decision rules

These machines describe lifecycle classes. Workflow-specific state names remain consumer data rather than a fixed universal enum.

2.8.1 Workflow run lifecycle

The run starts without active work and ends when a terminal state is reached.

STATE MACHINE Workflow run



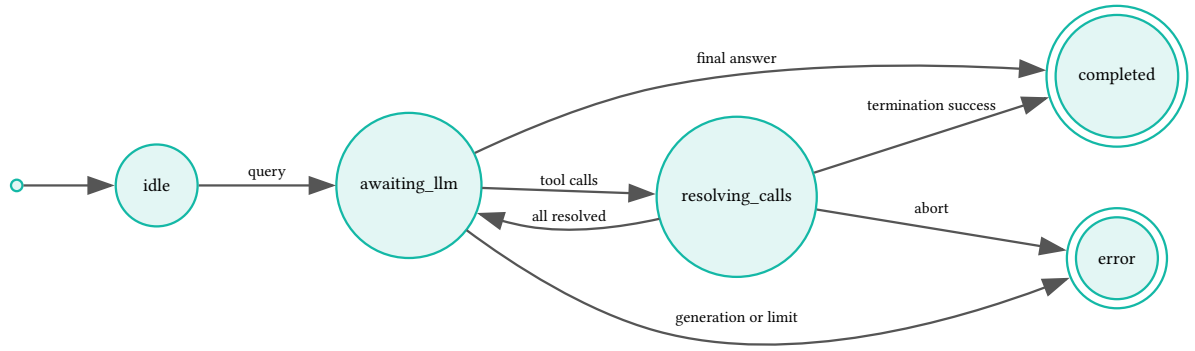
A suspended run retains its current Machine state. A rejected approval is a declared decision outcome; it can select any configured edge, not necessarily failure.

- Transition lookup uses exact state/outcome, wildcard state, wildcard outcome, then global wildcard. No match returns an error.
- Terminal states run no node. Defaults are done and failed, with done successful; overriding either terminal_states or success_states requires both, and success states must be a subset.

2.8.2 Orchestration run lifecycle

The run alternates model turns with call resolution.

STATE MACHINE Orchestration turn



resolving_calls abbreviates the precise awaiting states below; it is not a proposed runtime status.

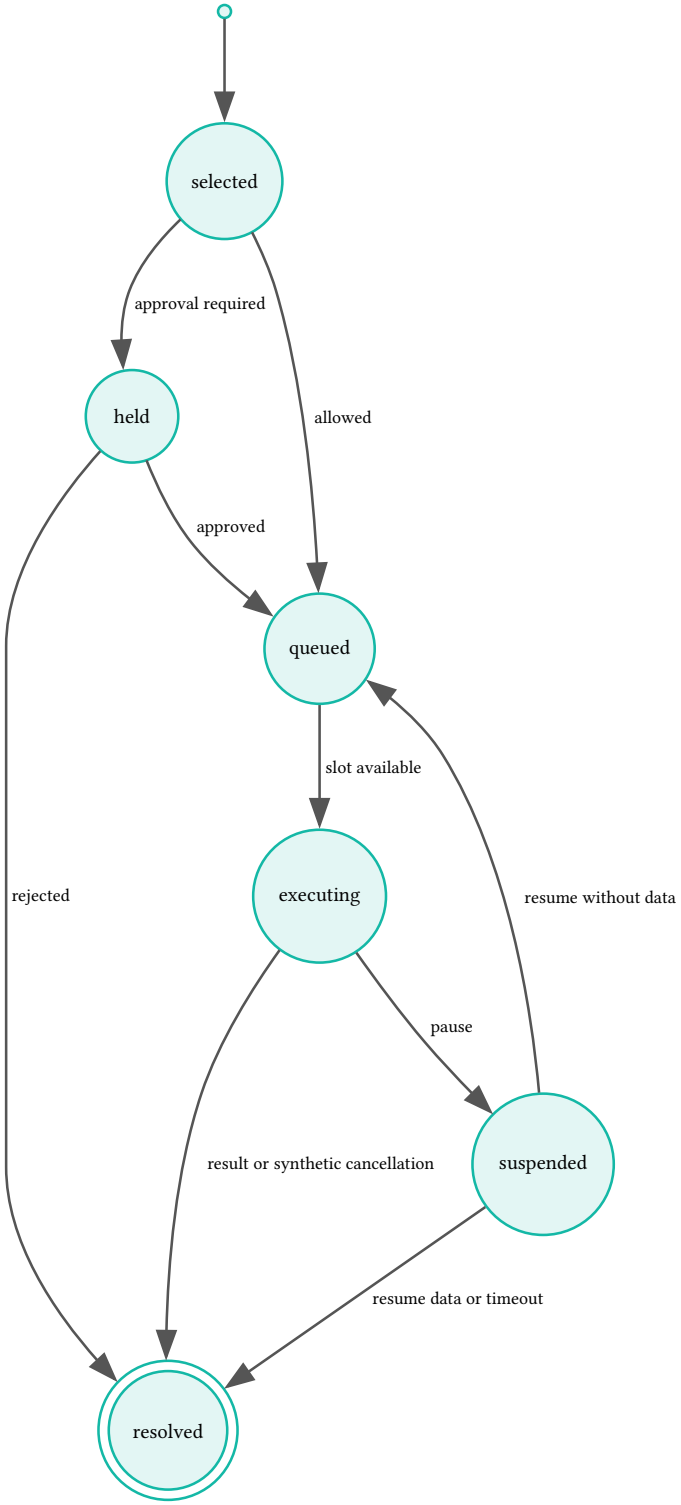
Runtime status	Meaning
awaiting_tools	Executing or queued calls remain
awaiting_approval	Only gated decisions remain
awaiting_tools_and_approval	Running calls coexist with gated decisions
awaiting_suspension	Only suspended calls remain
awaiting_tools_and_suspension	Running calls coexist with suspended calls

- StatusComputer derives the status from ToolConcurrency, ApprovalGate, and suspended_calls. In the observed implementation, approvals plus suspensions without executing calls produce awaiting_tools_and_approval; executing calls plus suspensions produce awaiting_tools_and_suspension even when approvals also remain. These labels do not expose every pending collection; the execution ledger retains the intended mixed-wait contract as an owner decision.
- Iteration limits are checked before termination-call interception. A final answer completes the run; a termination error is returned to the model so it can retry.

2.8.3 Tool call lifecycle

A call is held or queued before execution and resolves once its result has been accepted.

STATE MACHINE Tool call

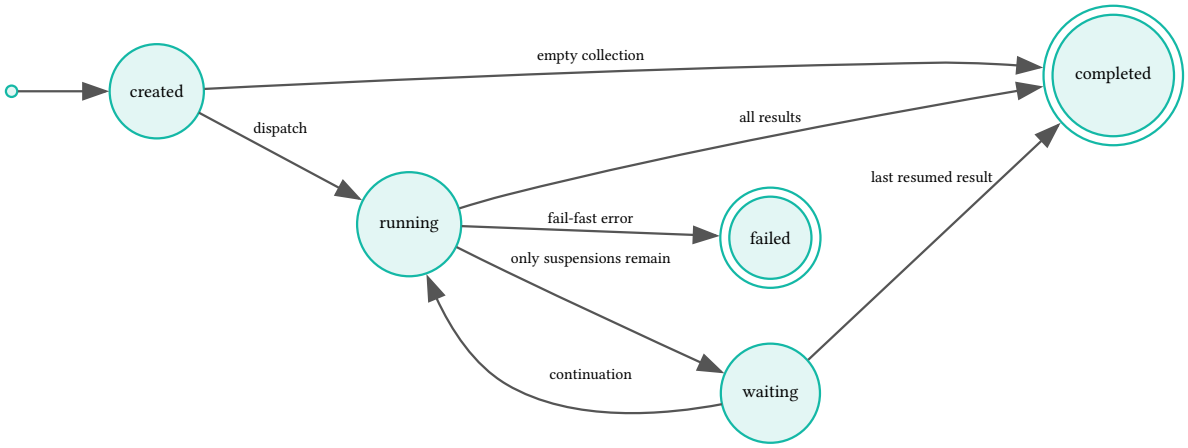


Failure, rejection, and sibling cancellation can resolve calls with synthetic results. Abort ends the owning run rather than requiring a successful call result. Withdrawal of an accepted approval is not specified.

2.8.4 Parallel execution lifecycle

An empty operation completes without dispatch; otherwise it waits for every required branch to resolve.

STATE MACHINE Parallel execution

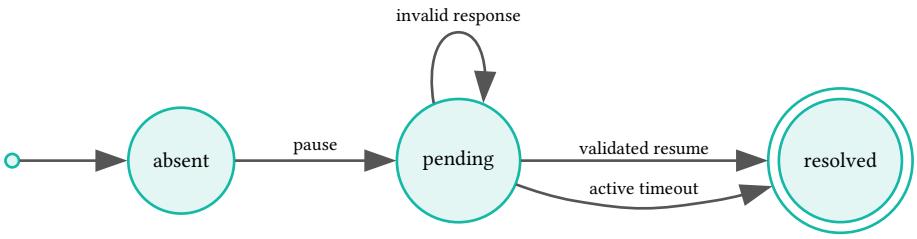


Queued and executing branches are distinct collections. A custom FanOut merge controls result aggregation; MapNode returns results in input order.

2.8.5 Suspension lifecycle

Pending membership records whether a pause can still consume a continuation.

STATE MACHINE Suspension

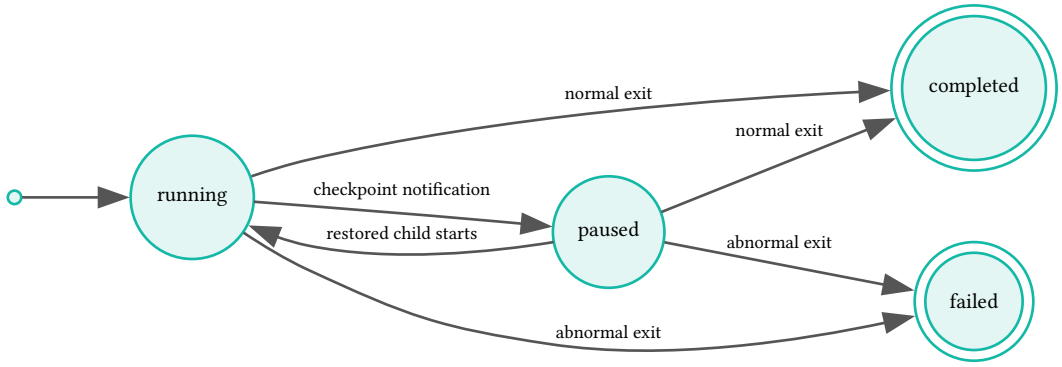


Once resolved, a late timeout cannot consume another transition. Cancellation and duplicate response behavior require the path-specific contracts in the execution chapter.

2.8.6 Child reference lifecycle

A parent tracks child progress separately from the process identifier used for transport.

STATE MACHINE ChildRef status



Children.record_result clears the communication phase without changing status. Children.record_exit marks normal exits completed and every other reason failed, including an exit recorded after pause. Hibernated remains a declared status without a distinct transition in these helpers; this observed lifecycle does not settle the intended checkpoint-exit policy.

Communication phase	Event or condition	Result
Absent	Child dispatch	spawning
spawning	Child-started confirmation	awaiting_result
awaiting_result	Accepted child result	Absent; status unchanged
spawning	Checkpoint replay	Re-emit SpawnAgent
awaiting_result	Checkpoint replay	No phase-owned replay; compatibility fallback can still apply

- Phase is independent of status. A missing child reference is the vacant state before dispatch; the diagram does not invent a spawning status on ChildRef. The execution chapter specifies compatibility fallback when phase maps and ChildRef fields disagree.

2.9 Effects and decision inputs

An accepted event changes the owning execution value; runtime operations apply its external consequences.

Change	New or queued work	In-flight work
Node configuration before query	Uses rebuilt nodes and tools	Mid-run reconfiguration is not specified
Approval required	Held call is not dispatched	Ungated siblings may continue
Approval rejection	Rejected call becomes synthetic result	Sibling policy continues, cancels, or aborts
Suspension	Continuation is held	Independent siblings can finish
Checkpoint	Restored configuration is rebuilt	Interrupted effects may be replayed
Accepted approval withdrawn	Unspecified	Unspecified

Decision point	Available inputs	Decision
Workflow transition	Current state, outcome, transition map	Next state or missing-transition error
Approval partition	Tool call, context, static metadata, policy	Proceed, hold, or policy error
Resume validation	Pending ID, response constraints, submitted data	Accept or validation error
Model response	Classified response and iteration count	Complete, invoke, retry, or fail
Branch completion	Pending, queued, suspended, and completed collections	Dispatch more, wait, merge, or fail

FIVE-SENTENCE ONBOARDING TEST

A consumer defines nodes and composition rules. A run carries context and uses those rules to select work. Each result

updates only its assigned working scope. A suspended operation keeps its identity until a validated continuation resolves it. The runtime executes effects and returns evidence to the owning run.

- Plain-language retelling: a parent gives a task to a participant and waits for its answer. The participant may ask a person for input, but it cannot rewrite the parent's protected input.

2.10 End-to-end walkthrough

The ETL example grounds configuration, result scoping, and outcome selection in one run.

- The consumer configures extract, transform, validate, load, and a rejection-handling state. The initial state is extract.
- Extract returns records. The owner stores the result under extract, then follows the ok edge to transform.
- Validate returns invalid. The declared invalid edge selects the rejection-handling node rather than treating invalid as a transport failure.
- The run reaches done and exposes accumulated context. If a node fails, its original reason follows the configured error edge or becomes a missing-transition error.

CONTEXT

Execution

3 Execution

3.1 Foundation

Execution connects the shared composition values to Jido agents, runtime directives, model calls, storage, and observations.

GOAL

Execute the same composition in supported runtimes

Consumers can drive directive loops directly for tests or use AgentServer for process-based execution.

GOAL

Resume nested work with preserved logical state

Checkpointing retains conversation, context, pending decisions, and child tracking while reconstructing runtime-derived fields.

NO-GOAL

Provide a host application or approval transport

Composer defines notification and resume data but does not supply a user interface, transport, durable scheduler, or authorization service.

NO-GOAL

Add unsupported collection or registry mechanisms

Custom MapNode reduction, incremental collection output, element-transform callbacks, service discovery, and a Jido AI skill adapter are not part of the migrated scope.

INVARIANT

Every incoming signal has an explicit route

AgentServer rejects unknown signal types; each strategy declares every signal it consumes.

mechanism

INVARIANT

A completed branch retains its collected result

A sibling suspension does not discard completed parallel results. Merge waits for all required branches to resolve.

mechanism

PRINCIPLE

Use existing provider and runtime abstractions

ReqLLM owns provider differences. Jido owns AgentServer, directive execution, signals, and base persistence.

PRINCIPLE

Keep assembly separate from execution

Skill assembly produces inspectable configuration without running a model loop.

3.2 Pending updates

ruling **Settle HumanNode exposure as an LLM tool** 2026-09-02

HumanNode.to_tool_spec returns nil and its test explicitly excludes LLM-tool use. The advisory-tool descriptions remain carried intent, not executable support; choosing whether to add that support still needs an owner decision.

ruling **Normalize approval policy and resume contracts** 2026-09-02

The observed policy gates only on the atom require_approval; a tuple with options does not gate. Both strategies use composer.suspend.resume, refuse unmatched resumes, and ignore resolved timeouts. The source's tuple-policy and legacy-response alternatives remain unselected.

ruling **Qualify the pure-strategy boundary** 2026-09-02

The implementation executes termination through Jido.Exec.run inline and emits observation effects from strategy helpers. These exceptions are confirmed; deciding whether to move them behind directives remains open.

ruling **Require safe checkpoint preparation and parent stripping** 2026-09-02

The default agent checkpoint retains `__parent__.pid` and does not invoke Composer preparation or add Composer schema and claim markers. The safe serialization requirement remains intent; automatic preparation and parent-reference cleanup are not implemented guarantees.

ruling **Choose whether to implement OTP hibernation** 2026-09-02

SuspendExec confirms log-only hibernate intent with unchanged state. The source's directive-return or passive AgentServer alternatives remain unselected; checkpoint-and-stop is a separate mechanism.

ruling **Settle persistence failures and replay safety** 2026-09-02

CheckpointAndStop stops and may notify its parent even when persistence is absent or fails. Resume thaws before its optional claim, leaves a failed delivery claim without rollback, and ignores final claim-update failure. These observed limitations do not endorse lost checkpoints or establish exactly-once effects; mixed-wait labels, ChildRef checkpoint-exit handling, and runtime policy restoration also remain owner decisions.

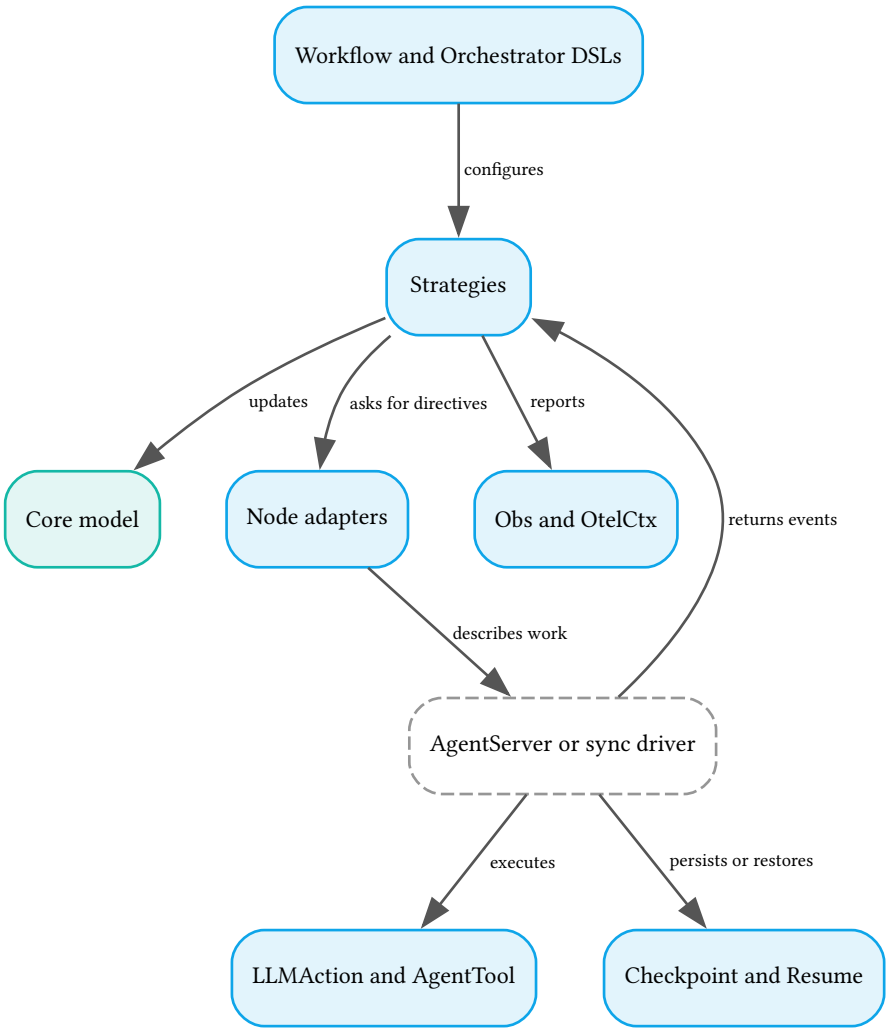
ruling **Verify carried error and MapNode contracts** 2026-09-02

Source intent requires uniform machine-readable error diagnostics and MapNode element-count, concurrency, and completion measurements. Current Error fields vary by class and several paths return ordinary errors; the observation code inspected does not establish the complete MapNode measurement contract. MapNode's direct path also supplies element-only parameters, unlike the full-context directive path; preserve the source's context intent until that difference is ruled.

3.3 System at a glance

The consumer API configures strategies. Strategies interpret events and return work for a runtime to perform.

ALTITUDE L3 · COMPONENTS Execution components



The core model is a separate documentation context. All blue components belong to execution; the runtime boundary performs actions, child creation, delivery, and persistence. The documented inline termination and observation exceptions are recorded above.

Consumer interface

Builds agents and starts or configures runs.

Node adapters

Translate configured operations into direct calls or directives.

Strategies

Accept results and update workflow or orchestration progress.

Suspension and persistence

Retain pending work and restore addressable continuations.

Observation and testing

Expose execution evidence and test each boundary.

- `execution` uses `core-model` as the supplier of shared contracts. Within execution, integration concerns remain sections rather than new bounded contexts.

3.4 Consumer interface

Both DSLs produce standard Jido Agent modules. Module names and descriptions become the metadata visible to a parent composition.

Entry point	Result	Purpose
Workflow.new	Agent	Initialize configured strategy state
Workflow.run(agent, context)	Agent and directives	Start a workflow without executing its effects
Workflow.run_sync(agent, context)	ok with context or error with reason	Drive to a terminal state for scripts and tests
Orchestrator.new	Agent	Initialize orchestration state
Orchestrator.query(agent, query, context)	Agent and directives	Start a model-directed run
Orchestrator.query_sync(agent, query, context)	ok with agent and result; suspended with agent and suspension; or error	Drive a synchronous query
Orchestrator.configure(agent, overrides)	Agent	Apply runtime configuration before query

```
Workflow.run(agent, context) -> {agent, directives}
Workflow.run_sync(agent, context) -> {:ok, result_context} | {:error, reason}
Orchestrator.query(agent, query, context) -> {agent, directives}
Orchestrator.query_sync(agent, query, context) ->
  {:ok, agent, result} | {:suspended, agent, suspension} | {:error, reason}
```

- Synchronous drivers are intended for scripts and tests, not to block inside AgentServer. Runtime composition uses directives and lifecycle signals.
- get_action_modules reads the currently configured modules; get_termination_module returns the configured termination action or nil.
- Jido.Composer is the namespace and documentation entry point, not a separate business-logic coordinator.

3.4.1 Workflow configuration

A workflow declares state bindings and an outcome-driven transition graph.

Option	Contract
name and description	Agent identity and parent-visible description
schema	Agent state schema; defaults to an empty list and passes to Jido.Agent
nodes	State atom to node specification
transitions	State-outcome pair to next-state atom
initial	Initial nonterminal state
terminal_states and success_states	Both absent or both supplied; successful states form a subset
ambient	Input keys protected as ambient
fork_fns	Named MFA transformations at child boundaries

Node specification	Interpretation
`ActionModule`	ActionNode with defaults
`{ActionModule, opts}`	ActionNode with per-use options
`{AgentModule, mode: :sync}`	AgentNode with explicit mode
`%FanOutNode{...}`	Explicit configured node struct

- Plain Action modules are wrapped as ActionNode; Agent modules are wrapped as AgentNode. Explicit node structs retain per-instance configuration.
- The compiler rejects missing initial nodes, invalid referenced nonterminal states, and invalid terminal/success configuration. It warns about unreachable states, nonterminal dead ends, reserved suspend edges, missing terminal targets, and incompatible optional I/O declarations.
- Terminal targets require no node binding. Defaults are terminal states done and failed, with done successful.

3.4.2 Orchestrator configuration

An orchestrator declares a set of operations and generation settings rather than an execution graph.

Option	Default	Meaning
name and description	Declared	Identity and parent-visible metadata
nodes	Required	Available action, agent, or explicit node specifications
model	nil	ReqLLM model identifier
system_prompt	nil	Instructions for model selection
max_iterations	10	Loop safety limit
temperature and max_tokens	nil	Provider defaults unless overridden
stream	false	Collect streaming generation internally
termination_tool	nil	Action for validated structured completion
llm_opts and req_options	Empty lists	Generation options and opaque HTTP options
max_tool_concurrency	Unbounded	Maximum executing calls per turn
ambient and fork_fns	Empty lists	Context extraction and boundary transformations
rejection_policy	continue_siblings	Effect of a rejected gated call
hibernate_after	Absent	Enable checkpoint selection for long suspensions

- Per-node requires_approval metadata is extracted into gated node names. Runtime approval_policy is evaluated for calls not already statically gated; it is not safely embedded as a closure in compile-time module attributes.
- configure accepts system_prompt, nodes, model, temperature, max_tokens, max_iterations, req_options, and conversation. It replaces supplied values; unknown keys raise ArgumentError.
- A node override rebuilds wrapped nodes, tool specifications, name_atoms, and schema_keys together. Termination action deduplication is re-applied; callers do not manually maintain internal maps.

> GROUNDING
> A caller reads get_action_modules, filters that list by the caller's role, and supplies nodes to configure before query. This is a host policy step; it is not evidence that Composer authenticates the caller.

3.5 Node adapters

Every node receives its own struct, flattened context, and options. Optional callbacks describe runtime dispatch, tool metadata, and I/O expectations.

Callback	Required	Contract
run/3	Yes	ok with result, ok with result and outcome, or error with reason
name/1 and description/1	Yes	Instance-visible identifier and description
to_directive/3	No	ok with directives and optional state effects
to_tool_spec/1	No	Name, description, parameter_schema map or nil
schema/1	No	Input schema or nil
input_type/1 and output_type/1	No	map, text, object, or any

```
run(node, context, opts) ->
  {:ok, result} | {:ok, result, outcome} | {:error, reason}
to_directive(node, flat_context, opts) ->
  {:ok, directives} | {:ok, directives, side_effects}
# side_effects is a keyword list applied by the strategy
```

Directive option	Purpose
result_action	Action atom routing execution results
meta	Call ID and tool name for ActionNode
tag	Parent-assigned child state or call identity
structured_context	Full Context value for child forking or parallel preparation

tool_args	Agent tool arguments merged into child input
request_fields	Flow-location enrichment for HumanNode
fan_out_id	Parallel operation correlation

- A two-element ok result implies outcome ok. An error result implies outcome error; a three-element ok result carries its explicit outcome.
- Strategy dispatch is polymorphic through the node's to_directive callback. The strategy applies returned state effects, including pending suspension or FanOut state, before emitting directives.

3.5.1 Action and agent adapters

`ActionNode` wraps a Jido Action; `AgentNode` wraps an Agent with per-use execution configuration.

Adapter	Configuration	Dispatch
ActionNode	Action module and options	Delegate metadata; emit RunInstruction
AgentNode	agent_module, mode, opts, signal_type, on_state	Direct sync helper or SpawnAgent
HumanNode	Question and response policy	Immediate suspend result
DynamicAgentNode	Registry and assembly_opts	Assemble and execute a task

- AgentNode defaults to sync. Direct run supports run_sync, query_sync, or the supported Jido AI ask_sync interface; async and streaming are not directly runnable and return an execution error.
- Runtime strategy dispatch may use SpawnAgent even for sync configuration. Async avoids blocking the parent; streaming permits selected upstream state notifications through on_state.
- FanOut and Map direct child execution calls run/3, so AgentNode children on that path must use sync. This limitation is part of the execution contract, not a failure of recursive composition.

3.5.2 Jido AI bridge

The bridge supports the documented ask_sync interface without exposing internal agent state as a tool schema.

- Detection requires ask_sync/3 and excludes modules exporting Composer run_sync/2 or query_sync/3. The bridge creates a temporary AgentServer, calls ask_sync, receives the result, and stops the server with a cleanup delay.
- The model sees a query string parameter. CoDAgent draft_sync, CoTAgent think_sync, and ToTAgent explore_sync are not detected by this interface.

3.5.3 Human input adapter

`HumanNode` constructs a request without blocking the caller.

Field	Default	Meaning
name and description	Required	Identifier and purpose
prompt	Required	Static text or context-to-text function
allowed_responses	approved and rejected	Permitted outcome atoms
response_schema	nil	Optional structured response validation
timeout	infinity	Wait duration in milliseconds
timeout_outcome	timeout	Expiry outcome
context_keys	nil	All keys unless a subset is selected
metadata	Empty map	Notification metadata

- The node evaluates its prompt, selects visible context, constructs ApprovalRequest, stores it under `__approval_request__`, and returns ok with suspend. The strategy interprets the marker and owns waiting.
- In workflows, the human-input state and the guarded action are separate states. approved means the person approved, not that the guarded action succeeded.
- HumanNode.to_tool_spec returns nil, so direct LLM-tool exposure is unsupported. The source's advisory HumanNode tool remains an unresolved alternative, distinct from implemented mandatory approval gating.

3.5.4 Parallel and traverse adapters

`FanOutNode` runs named branches; `MapNode` maps one child over a runtime collection.

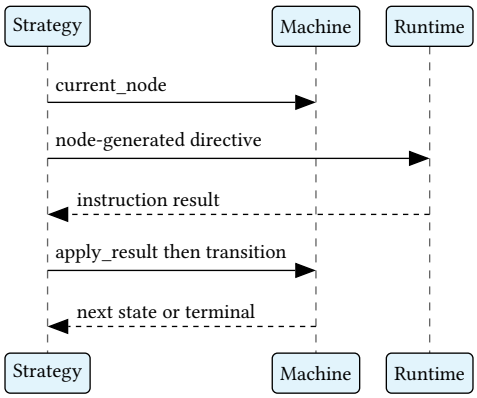
Field	FanOutNode	MapNode
name	Required identifier	Required identifier
child selection	Named branch/node pairs	over key or key path and one node
merge	deep_merge or custom function	Ordered results list
timeout	30,000 ms default; infinity allowed	30,000 ms per element
max_concurrency	nil means unbounded	nil means all
on_error	fail_fast or collect_partial	fail_fast or collect_partial; fail_fast default

- FanOut default merge scopes each result by branch name and deep-merges the resulting map. A custom merge receives branch-name/result pairs; collect_partial retains errors alongside successes for that merge.
- FanOutNode.to_tool_spec returns nil. It is not directly exposed as an LLM tool; an enclosing agent composition can provide a tool boundary.
- MapNode reads over at dispatch time. Its directive path merges map elements into base context and wraps other elements under item before merging. Direct run instead passes only the element map or item wrapper; the source's full-context intent remains pending for that path.
- MapNode preserves input order and returns results under results, subsequently scoped by the parent. Empty, missing, and non-list collections produce an empty results list without element dispatch.
- Both directive paths create FanOutBranch with child_node and params. Indexed Map branches use generated item identifiers; the executor calls the child node's run/3 uniformly.
- Backpressure queues excess branches and refills slots when results arrive. Fail-fast cancels running agent children through StopChild; MapNode collect_partial retains successful values and error tuples in input order.
- Nested traverse needs no special mechanism because MapNode is a node. Custom reduction belongs in a following ActionNode; streaming collections and element-transform callbacks remain outside the migrated scope.

3.6 Workflow strategy and runtime protocol

The strategy drives Machine operations and retains enough correlation state to interpret asynchronous results.

SEQUENCE One workflow node



The returned instruction result is interpreted before the next directive is chosen. A suspend result takes the waiting path instead of transition lookup.

Strategy field	Meaning
machine	Current Machine value
module	Owning strategy module
error_reason	Original failure or nil
pending_child	Tag/node pair or nil
child_request_id	Child correlation identifier or nil
pending_suspension	Active Suspension or nil
fan_out	Parallel collection state or nil

children	Serializable refs and phases
`_obs`	Workflow observation state

Machine operation	Contract
new/1	Initialize from options
current_node/1	Read the node bound to current state
transition/2	Return updated machine or transition error
terminal?/1	Test whether execution has ended
apply_result/2	Adapt and scope the accepted result

3.6.1 Workflow signal routes

Every listed signal maps to one strategy action; action results arrive as Jido.Instruction payloads rather than arbitrary tuples.

Signal	Action
composer.workflow.start	workflow_start
composer.workflow.child.result	workflow_child_result
jido.agent.child.started	workflow_child_started
jido.agent.child.exit	workflow_child_exit
composer.fan_out.branch_result	fan_out_branch_result
composer.suspend.resume	suspend_resume
composer.suspend.timeout	suspend_timeout
composer.child.hibernated	child_hibernated

- workflow_start initializes context and dispatches the initial node. workflow_node_result and workflow_child_result scope a result, extract its outcome, transition, and dispatch the next node.
- child_started registers lifecycle metadata in Children; child_exit handles unexpected termination or cleanup. Startup context is carried in SpawnAgent options; older narrative references to sending a second startup signal are not an additional required step.
- fan_out_branch_result stores a branch result and either dispatches queued work, waits for suspended work, or merges and transitions. child_hibernated records a paused ChildRef and checkpoint key.

3.6.2 Error preservation

Failures retain their diagnostic value from the original node through nested callers.

Source error class	Intended trigger	Implemented class
Validation	Invalid DSL configuration, state references, or transition keys	invalid
Transition	No matching state/outcome transition	transition
Execution	Node or child operation failure	execution
Communication	Delivery, timeout, or unexpected child exit	communication
Orchestration	Generation, tool selection, or iteration failure	orchestration

- The source requires machine-readable error codes, human-readable messages, and diagnostic context identifying current state, node, and outcome. The intended mapping above preserves that obligation; it does not assert that every failure path constructs that class.
- Error implements Splode classes and constructors with messages and details. TransitionError adds state and outcome; ExecutionError adds node; the remaining constructors do not expose all three fields.
- Class atoms and exception modules identify implemented error kinds; the constructors do not declare a separate uniform code field. Ordinary RuntimeError directives and preserved underlying reasons also remain in use; uniform diagnostic coverage stays in the pending ledger.
- Capture error_reason from node and child errors, failed transitions, FanOut fail-fast, and failed timeout or resume transitions. Initialize it to nil on a fresh run.
- A successful error transition can route to a configured recovery state. A missing transition produces an Error directive; terminal failure exposes the captured reason.

- The DSL reads error_reason with Map.get and uses workflow_failed only when the key is absent. An existing nil value remains nil; the source's broader fallback wording is not implemented.
- Failure paths close the agent span and record the actual error. Direct-failure helpers set status, preserve the reason, and close the span together.
- Original errors survive checkpoints when serializable. A child workflow's returned reason becomes its parent's error_reason rather than an inspected string.

3.7 Orchestrator strategy and model boundary

The strategy manages turn state and tool resolution. LLMAction performs generation; AgentTool adapts node metadata and results.

3.7.1 Turn state and routes

Orchestration state contains configuration, accumulated conversation, and independent collections for executable, gated, and suspended calls.

State group	Fields
Generation configuration	model, system_prompt, temperature, max_tokens, stream, llm_opts, req_options
Operation configuration	nodes, tools, termination_tool_name, termination_tool_mod
Progress	status, iteration, max_iterations, conversation, result
Call tracking	tool_concurrency, approval_gate, suspended_calls
Composition state	context, ambient_keys, children, pending_suspension, hibernate_after, `_obs`

Signal	Action
composer.orchestrator.query	orchestrator_start
composer.orchestrator.child.result	orchestrator_child_result
jido.agent.child.started	orchestrator_child_started
jido.agent.child.exit	orchestrator_child_exit
composer.suspend.resume	suspend_resume
composer.suspend.timeout	suspend_timeout
composer.child.hibernated	child_hibernated

- orchestrator_start seeds context and conversation, then emits RunInstruction for LLMAction. orchestrator_llm_result completes, fails, or dispatches selected calls.
- Action calls return through orchestrator_tool_result; agent calls return through orchestrator_child_result. Each normalized result carries id, name, and result.
- ToolConcurrency contains pending, completed, queued, and max_concurrency. StatusComputer combines it with gated calls and suspended calls before allowing another model turn.
- Context splits declared ambient keys from input and inherits the reserved ambient marker at child boundaries. Action nodes see a flat map; agent nodes receive forked context. Raw ambient values are not inserted into model messages.

3.7.2 Tool queue capacity

ToolConcurrency retains call IDs awaiting results, completed result records, and queued call data independently.

Operation or field	Observed contract
max_concurrency	nil is unbounded; otherwise limits executing calls
max_queue_depth	nil is unbounded; otherwise limits enqueue/2
enqueue/2	Returns ok with updated state or error with queue_full at the limit
drain_queue/1	Moves queued calls into free slots and returns those calls for dispatch
dispatch/3	Replaces pending and queued collections and clears completed results

- max_queue_depth belongs to ToolConcurrency.new options, not to the public orchestrator DSL or configure options. Strategy initialization supplies max_concurrency only.
- The bound is checked by enqueue, not by dispatch's initial queue assignment. When an approved call cannot be enqueued, the strategy removes its gate entry and records a tool-error result rather than running it.

3.7.3 AgentTool boundary

AgentTool converts node descriptions into model-facing tool specifications.

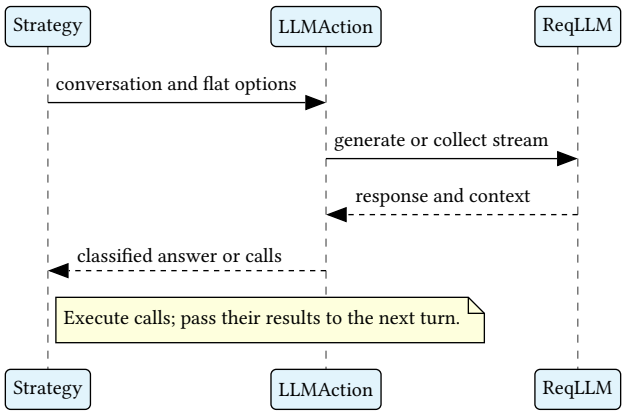
Operation	Result
to_tool(node)	ReqLLM.Tool from node name, description, and schema
to_context(tool_call)	Argument map suitable for node execution
to_tool_result(call_id, name, result)	Correlated normalized result for the next turn

- NimbleOptions and Zoi schemas are converted into JSON Schema. The ReqLLM tool carries a no-op callback because Composer dispatches execution externally.
- Call arguments are parsed maps. The call ID is retained independently of the tool name; repeated calls can share a tool name but must still correlate individually.
- name_atoms and schema_keys are derived from configured operations. The migration does not add a new dynamic atom-conversion policy beyond the existing documented maps.

3.7.4 LLMAction boundary

LLMAction calls ReqLLM directly; no separate provider facade or configurable LLM behavior is introduced.

SEQUENCE Generation and tool feedback



Provider-specific message construction remains inside ReqLLM.

- The first call constructs a ReqLLM.Context from prompt and query. Later calls append tool results with ReqLLM.Context.tool_result; response context preserves assistant messages and mixed content.
- stream false uses generate_text/3. stream true uses stream_text/3 and collects internally before classification; it does not promise incremental strategy-level results.
- Instruction parameters are conversation, tool_results, tools, model, query, system_prompt, temperature, max_tokens, stream, llm_opts, and req_options. The action merges explicit generation settings into llm_opts and maps req_options to req_http_options.
- Response classification returns tool_calls, optionally with reasoning text, or final_answer. Reasoning accompanying tool calls may be logged or discarded; it does not change dispatch semantics.
- ReqLLM owns provider encoding, argument parsing, result formatting, and response context. Strategy state owns the retained conversation and supplies it to the next call.

3.7.5 Structured termination

A termination action provides validated structured completion without adding a separate generation mode.

- The action's metadata and schema become a tool visible to the model. Its name is registered for lookup, but it is not a regular node and does not receive ordinary context scoping or approval gating.
- The max-iterations guard runs first. The first termination call then takes precedence over regular sibling calls; siblings in that batch are dropped.
- The documented implementation executes the termination action through Jido.Exec.run inline. Success completes with NodeIO.object; failure becomes a model-facing error result and the loop continues.
- A consumer requiring approval around a sensitive termination action cannot assume ordinary gated-node metadata applies. Moving execution behind directives would be a design change, not part of this migration.

3.7.6 Failures and restoration

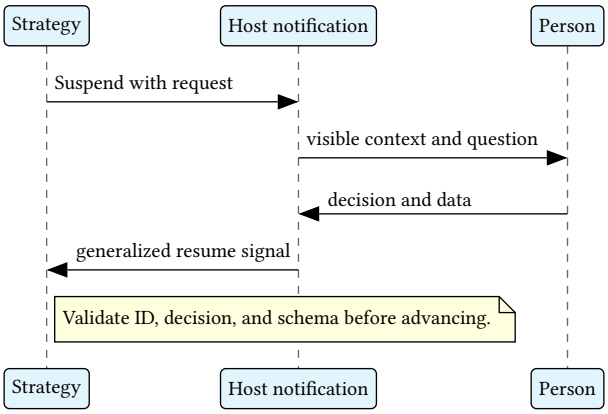
Failure and restore paths preserve structured state rather than replacing it with display strings.

- Generation failures and iteration exhaustion end the run. Tool errors may be presented to the model as results; explicit abort policies end execution.
- The final error remains structured in result for callers. Only LLM-facing text uses inspect-style formatting. Error paths close iteration and agent spans, including partition errors and abort_iteration rejection.
- Fresh init applies when no matching state exists or status is idle. Restore applies to matching non-idle strategy state and preserves conversation, pending calls, iteration, context, and results.
- Restore rebuilds nodes, tools, name maps, gated-node metadata, approval policy, maximum concurrency, and req_options from runtime configuration. The provenance of a runtime-only approval-policy closure after a restart remains a pending decision.

3.8 Suspension and approval protocols

A pause is a valid result requiring continuation. Its owner validates correlation and policy before accepting resumed work.

SEQUENCE Human continuation



The host owns delivery and respondent authentication. The record itself contains no authorization proof.

- Suspension reasons are human_input, rate_limit, async_completion, external_job, and custom. The record carries ID, created_at, resume_signal, timeout, timeout_outcome, metadata, and optional ApprovalRequest.
- Suspend carries suspension, notification configuration, and hibernate intent. SuspendForHuman is the compatibility constructor for human_input, not a separate lifecycle.
- ApprovalRequest contains ID, prompt, visible_context, allowed_responses, response_schema, timeout, timeout_outcome, metadata, and created_at. Strategy enrichment adds agent_id, agent_module, workflow_state or tool_call, and node_name.
- ApprovalResponse contains request_id, decision, data, respondent, comment, and responded_at. The strategy validates identity, allowed decision, and data schema before recording hitl_response in context.

3.8.1 Workflow pause and resume

The workflow keeps its Machine at the current state while its strategy waits.

- Extract `__suspension__` or legacy `__approval_request__`. Wrap a legacy request as human_input, enrich its flow location, merge remaining result data, store pending_suspension, and set waiting.
- Emit Suspend and an optional Schedule timeout. A qualifying long pause can additionally emit CheckpointAndStop.
- Resume validates the pending ID. Human input performs request-specific validation; other reasons use outcome and data from the signal. Accepted data is merged, pending state is cleared, and the selected outcome advances the Machine.
- An active timeout consumes timeout_outcome, defaulting to timeout. A timeout for an already-resolved suspension is a no-op.

3.8.2 Approval gate and sibling policy

The gate partitions model-selected calls before their ordinary execution.

Rejection policy	Effect
continue_siblings	Let running calls finish and return all results to the model
cancel_siblings	Stop running child calls and synthesize cancellation results
abort_iteration	Cancel remaining work and fail the run

- Static requires_approval and dynamic approval_policy both feed the partition. Static membership gates without calling the dynamic policy; otherwise only the exact atom require_approval gates.

- A tuple such as require_approval with options is not accepted as a gating result and proceeds ungated. This compatibility limitation is not a recommended policy return.
- A held call receives an approval request; ungated calls need not wait. Rejection produces a synthetic tool result explaining the refusal; unless the policy aborts, the model may choose a different approach or return a final answer.
- Status is awaiting_tools_and_approval while siblings execute, then awaiting_approval when only held decisions remain. All required collections must resolve before the next turn; mixed approval/suspension labels follow the core lifecycle chapter's observed precedence.

3.8.3 General tool suspension

A tool can suspend without human approval, such as while awaiting a rate limit or external operation.

- Detection accepts a direct suspend status or an ok status carrying suspend in effects. The strategy extracts or constructs Suspension, removes the call from pending, stores suspension plus call under suspended_calls[id], and emits Suspend.
- Resume with data accepts that data as the result without executing the tool again. Resume without data re-dispatches the call; both remove the old suspension entry. Unknown IDs produce an Error directive in the documented orchestrator path.
- An active timeout becomes an error tool result and removes the entry. Once pending, gated, and suspended collections resolve, the next model turn receives all collected results.

3.8.4 Parallel and nested pauses

Parent waiting does not transfer ownership of a child's approval decision.

- FanOut tracks queued, executing, completed, and suspended branches independently. When running work finishes and only suspensions remain, it emits Suspend for each unresolved branch. Resumed results join completed_results before final merge.
- An outer workflow waiting for InnerOrchestrator remains waiting while the child handles ResearchAgent and a gated DangerousAction. Research can finish before approval; the outer workflow sees only the child's eventual result or error.
- Independent children can hold distinct approval requests concurrently. A sequential Machine has one current state, but that state can contain parallel children with separate requests.
- Inner and outer timeouts are independent. If an outer timeout stops a child first, the host should mark the corresponding external request cancelled; the source does not define a durable notification protocol for that cancellation.
- A response arriving during checkpoint transition must be queued by the host for post-restore delivery. The strategies ignore resolved timeouts but emit Error directives for unmatched resumes; the external Resume helper instead returns no_matching_suspension.

3.9 Checkpointing and resumption

Durable checkpoints retain logical execution state while runtime handles and observation spans are reconstructed or reset.

Mode	Trigger	Effect
Live wait	Suspension begins	Process and full heap remain
Hibernate intent	Suspend.hibernate true or after setting	Current documented integration logs intent; memory unchanged
Checkpoint and stop	Suspension timeout meets hibernate_after threshold	Persist state, notify parent, stop process

- The detailed persistence contract describes threshold selection from suspension timeout, not a guaranteed elapsed-time timer. Older narrative says the threshold fires; that wording must not imply a scheduler that is not defined.
- CheckpointAndStop resolves storage from its directive or agent lifecycle configuration, persists through Jido.Persist.hibernate, emits composer.child.hibernated, and returns a stop tuple. checkpoint_data is reserved and currently described as unread.
- The runtime accepts ok, async, and stop directive-executor returns. The legacy OTP-hibernate proposal would require either a new return variant or a passive AgentServer option; neither is claimed available.

> **STOPPING DOES NOT PROVE A CHECKPOINT EXISTS**
> CheckpointAndStop logs missing storage or persistence failure, then continues to parent notification when a parent PID exists and returns the hibernated stop tuple. A notification therefore does not prove recoverability. This observed behavior remains a safety limitation, not approval to discard state after failed persistence.

3.9.1 Checkpoint data and serialization

Jido's default agent checkpoint and Composer's optional strategy preparation are separate paths.

Field or marker	Location	Observed behavior

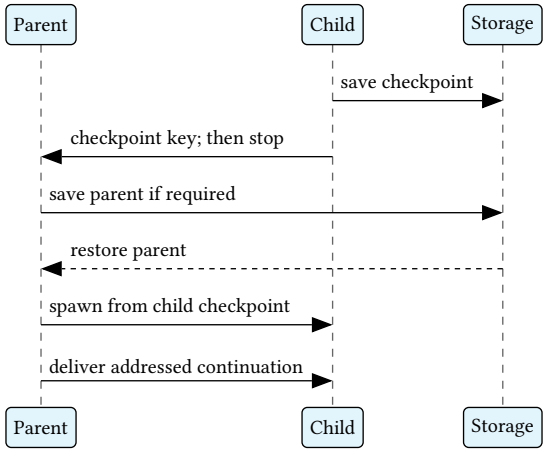
version	Base checkpoint	1
agent_module and id	Base checkpoint	Restorable agent identity
state	Base checkpoint	Includes `__strategy__`
thread	Added by Jido.Persist	Thread ID and revision or nil; thread content is stored separately
children	Strategy state	Child references and phases
checkpoint_status	Strategy state after Composer preparation	Set to hibernated only when absent
checkpoint_schema	Source-intended checkpoint field	Not inserted by the default callback or Composer preparation; schema_version/0 separately returns composer_v1
Claim status	Optional host storage callback	Not a default top-level checkpoint field

- The preservation requirement covers Machine state, context, history, conversation, iteration, pending suspensions, parallel results, queued branches, gated calls, and suspended calls. The default callback copies agent state; it does not invoke the strategy's prepare_for_checkpoint hook.
- Explicit Composer preparation delegates to strip_for_checkpoint, resets Obs, and removes supported top-level functions and the nested approval policy. It also inserts checkpoint_status; callers must not infer that Jido.Persist.hibernate or CheckpointAndStop performs this step automatically.
- The current storage format is compressed Erlang term serialization. JSON and MsgPack are possible future export formats, not equivalent supported persistence promises.
- The source requires stripping `__parent__.pid` while preserving id, tag, and meta. The observed default callback retains that PID; Composer's strategy-only preparation cannot remove it from the containing agent state.
- Span handles and arbitrary nested custom closures are not proven checkpoint-safe. Safe parent stripping and automatic transient-state cleanup remain requirements in the pending ledger, not completed guarantees.
- Fork transformations use MFA tuples to remain serializable. Restore rebuilds configured nodes and policies from strategy options; a runtime-only policy without a durable reconstruction source can be lost.

3.9.2 Nested checkpoint and restore order

Checkpointing proceeds from child to parent; restoring an inactive tree proceeds from parent to child.

SEQUENCE Nested checkpoint and restore



A live parent need not be restored again. Targeted resume addresses the suspended agent; fresh parent links are needed when the enclosing tree was stopped.

- The parent records the child's checkpoint key and paused status before its own checkpoint. ChildRef retains agent module, agent ID, tag, suspension ID, checkpoint key, status, and phase.
- The source's parent-first restore order is a host integration responsibility. Resume does not discover the outer tree or start AgentServer; it returns paused-child SpawnAgent directives after the directives produced by delivery.

```
Resume.resume(agent_or_nil, suspension_id, resume_data, opts)
  -> {:ok, resumed_agent, directives} | {:error, reason}
# Required: deliver_fn: fn agent, {:suspend_resume, params} ->
#   {updated_agent, directives}
# For nil agent: thaw_fn: fn agent_id -> {:ok, agent} | {:error, reason}
```

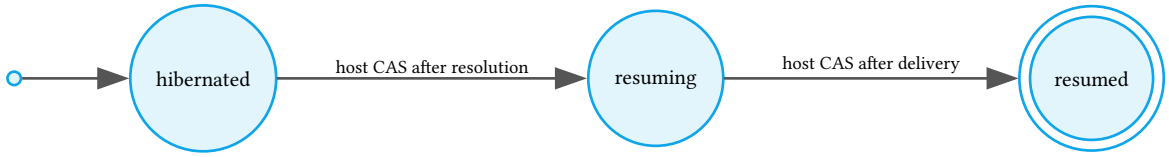
```
#               agent_id: id
# Optional: storage: %{compare_and_set_status: fn id, from, to -> result end}
```

- The helper accepts an existing Jido.Agent value, or calls thaw_fn with agent_id when that value is nil. Registry lookup, storage selection, and process startup belong to caller-supplied integration.
- deliver_fn receives a strategy-command tuple, not a Jido.Signal. A host targeting AgentServer translates it to the composer.suspend.resume route and owns delivery.
- Matching checks cover pending_suspension, suspended_calls, and suspended FanOut branches. ApprovalGate entries are handled by the orchestrator's resume route but are not matched by this external helper.
- Missing agent resolution returns agent_not_available; no matching suspension returns no_matching_suspension. Successful helper delivery returns the agent and directives; an Error directive from the strategy is not automatically converted into a helper error tuple.

3.9.3 Resume claim and replay

Optional host storage can claim delivery after the helper has resolved or thawed the agent.

STATE MACHINE Checkpoint claim



This is the permitted status-transition graph, not proof that both writes succeed. The source requires competing claims to be refused; the host's callback supplies that atomicity and its error. Without storage, Resume skips both claim calls.

CLAIMS DO NOT ESTABLISH REPLAY SAFETY

Thaw occurs before the first claim. A failed delivery has no claim rollback, and failure of the final claim update is ignored while Resume returns its success tuple. Repeated effects, stranded claims, and recovery from interrupted delivery remain unresolved safety requirements.

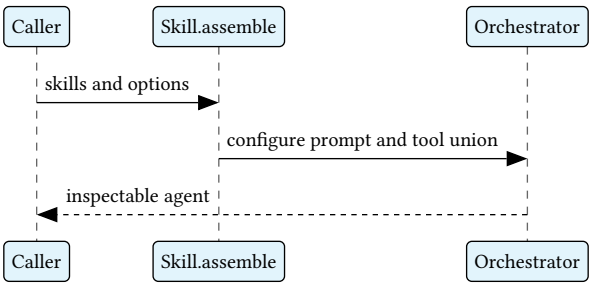
Persisted condition	Replay
Child phase spawning	Re-emit SpawnAgent
Child phase awaiting_result	Keep waiting
Orchestrator awaiting_llm	Re-emit model call from conversation
awaiting_tools or legacy awaiting_tool	Re-dispatch pending calls
awaiting_tools_and_approval	Replay pending calls; gated calls remain held
awaiting_tools_and_suspension	Replay pending calls; suspended calls retain resume path

- Checkpoint.replay_directives combines child phase replay with the strategy's replay_directives_from_state callback. For a Children struct, no spawning tags triggers fallback to ChildRef.phase even when the phases map is nonempty; the legacy flat-map path falls back only when child_phases is empty.
- A stale spawning phase on ChildRef can therefore re-emit SpawnAgent despite another phase in Children.phases. This observed compatibility precedence is not a promise of duplicate-free replay.
- Resume prepends replay directives when checkpoint_status is present, then appends delivery directives and paused-child respawns. The default checkpoint path does not add that marker; callers must explicitly arrange the required preparation.
- In-flight HTTP and signals are not durable. Repeated dispatch can repeat external effects unless the host or operation enforces idempotency.
- New fields obtain restore defaults; removed fields are ignored. Changed state graphs require the agent's restore callback to map old states. If a required module was renamed or removed, restore fails with a clear error until an explicit migration is supplied.
- Schedule timers do not survive process death. The host must find expired suspensions using created_at and timeout, address live or checkpointed agents, and mark missing checkpoints expired.

3.10 Skill assembly and dynamic delegation

A capability bundle becomes an ordinary orchestrator configuration before the caller chooses how to execute it.

SEQUENCE Assembly without execution



No model call is required to inspect the assembled prompt and operations.

- Skill fields are name, description, prompt_fragment, and tools. Tool modules follow the same action/agent wrapping rules as orchestrator nodes, including workflow, orchestrator, and supported Jido AI agents.
- assemble combines base_prompt, a Capabilities heading, and selected prompt fragments in order. It deduplicates tool modules and uses ordinary orchestrator configuration to instantiate the agent.
- Assembly options include base_prompt, model, and max_iterations with default 10; additional generation and HTTP options pass through configure where supported. Prompt concatenation does not infer fragment dependencies or remove semantic duplication.
- DynamicAgentNode stores name, description, skill_registry, and assembly_opts. It exposes task text and skill-name list, looks up selected skills, assembles an agent, calls query_sync, and returns the result.
- A workflow may call assembly separately and inspect its product before execution. Predefined specialists remain appropriate for stable configurations; runtime assembly serves combinations selected from a larger capability set.
- The registry is an inline list. Module registries, external discovery, a custom prompt composer, and a Jido AI Skill adapter remain extensions, not promised current behavior.

```
Skill.assemble(skills, opts) -> {:ok, agent} | {:error, reason}
DynamicAgentNode.run(node, context, opts)
  -> {:ok, result} | {:error, reason}
```

- Skill.assemble rejects unknown options with an ArgumentError inside an error tuple; it does not raise that validation error.
- DynamicAgentNode resolves skill names in requested order and returns an error naming the first absent skill. Assembly errors and query errors propagate; a suspended child query becomes an error containing the suspension rather than a resumable node suspension.
- DynamicAgentNode.to_directive merges tool_args over flat context and emits RunInstruction for ExecuteAction, retaining result_action and meta. ExecuteAction invokes run/3 and wraps success under result; direct run returns the unwrapped result.

> **GROUNDING**
> Selecting web_search and planning combines their instructions and deduplicates shared tools. DynamicAgentNode keeps model and base_prompt inside assembly_opts; the older use-case sketch placing those fields directly on the node is not its canonical configuration.

3.11 Observation and diagnostics

Observation records execution evidence without granting authority over results, approvals, or workflow transitions.

ALTITUDE L3 · COMPONENTS Observation ownership

Obs tracks values; Jido.Observe and OtelCtx perform instrumentation effects. Optional OpenTelemetry absence is a deliberate no-op, not a failed business operation.

Span	Start	Finish
Agent	Workflow or orchestrator start	Terminal success or error
Iteration	Before model call	All tool results, final answer, or error
LLM	Before generation directive	Model result
Tool or node	Dispatch	Node or child result
Parallel branch	Child branch execution	Child return

- Telemetry prefixes share jido, composer and then agent, iteration, llm, or tool. Workflow nodes also use tool so one tracer mapping covers both composition patterns.
- Orchestrator.Obs retains agent_span, llm_span, tool_spans, iteration_span, and cumulative_tokens. Workflow.Obs retains agent_span and node_span.
- Jido.Observe.start_span emits start and returns opaque context; finish_span emits stop with duration; finish_span_error emits exception. A pluggable tracer receives callbacks.

- AgentObs.JidoTracer maps agent, llm, and tool directly and iteration to chain, then uses the Phoenix translator for OpenInference attributes.

3.11.1 Nesting and sibling isolation

Child spans must inherit the caller's tool span without making sibling calls appear nested.

- Save the previous OTel context, start each tool span, capture its context by call ID, and restore the previous context. Pass the captured value through SpawnAgent.meta.otel_parent_ctx.
- The synchronous driver attaches it with OtelCtx.with_parent_context around child execution. A child workflow can reattach parent metadata before opening its agent span. Parallel tasks capture before spawn and attach inside each task.
- with_parent_context restores through try/after. get_current returns nil and attach does nothing if OpenTelemetry.Ctx is unavailable.

3.11.2 Measurements and checkpoint hygiene

Measurements describe execution outcomes and are reset where their runtime references cannot persist.

Span	Measurements
Agent	result, status, iterations, cumulative prompt/completion/total tokens, error
LLM	token usage, finish_reason, normalized output messages
Tool	tool_name, result, ok or error status, error
Iteration	Outcome summary or error

3.11.3 MapNode observation requirements

The traverse source requires one parent node span for MapNode and child operations for its element executions.

Required observation	Purpose
Element count	Size of the selected collection
Concurrency	Parallel execution setting
Completion status	Whether collection completed or failed
Parent and child spans	Relate the collection to each element execution

- The observed workflow path opens the ordinary node span; MapNode's direct task path captures and reattaches OTel context for element work. These mechanisms do not establish that all required measurements are emitted.
- The inspected MapNode and workflow observation code does not emit explicit element-count and concurrency measurements for the collection span. The full source requirement is preserved in the pending ledger rather than removed or claimed complete.
- NodeIO is unwrapped for output measurements. The reserved tuple ambient marker is removed from tool arguments, child results entering the LLM, and workflow result measurements because those encoders cannot represent it.
- Explicit Composer checkpoint preparation resets Obs; the default persistence path does not guarantee that reset. Snapshot exposes status, done?, result, and details without requiring callers to inspect strategy internals.
- A paused snapshot can expose request_id, node_name, reason, and waiting_since. These are diagnostic data, not a substitute for validating the addressed resume.

3.12 Applied design and host responsibilities

The existing design selects an Elixir library and delegates runtime mechanisms to established Jido and ReqLLM boundaries.

Concern	Applied choice	Owner and consequence
Language and runtime	Elixir on BEAM	Consumer library; immutable values and OTP process integration
Agents and effects	jido, jido_action, jido_signal	Jido owns AgentServer, instructions, directives, and signals
Model providers	req_llm over Req	LLMAction calls the provider-independent integration directly
Schemas	Zoi; legacy NimbleOptions	Node and DSL validation; AgentTool JSON Schema conversion

Errors	Splode	Validation, transition, execution, communication, orchestration classes
Accumulation	deep_merge	Composition layer scopes results before merging
Serialization	Erlang terms; Jason for JSON	Persist storage and model-facing schema/result boundaries
Observation	telemetry and optional OTel	Jido.Observe, Obs, OtelCtx, and host tracer
Provider tests	ReqCassette and Req.Test	Test-owned HTTP interception
Checkpoint storage	Jido.Persist and configured adapter	Host supplies backend and verifies atomic claim support

- The extracted decisions are ADR-0001, ADR-0002, ADR-0003, and ADR-0004. They preserve source rationale; they do not select new versions or claim current provider availability.
- The library is not a Phoenix application and does not mandate a database. Durable timeout scheduling, storage retention, notification retries, and credential management belong to the host integration.
- Prompt management consists of the declared system prompt, runtime override, and Skill fragments. Retrieval-augmented generation and MCP are not specified as built-in subsystems; ordinary nodes may wrap such capabilities.
- Approval and RBAC filtering require host identity and authorization. The documents do not specify authentication for resume, encryption, retention duration, sensitive-result redaction, or model-input injection defenses; callers must not infer those guarantees.
- Jido.Exec.Chain is a single-process linear action pipeline with shallow merge. Composer adds outcome graphs, agent composition, and scoped accumulation; Jido.Plan is a dependency DAG, while Workflow is an event-driven FSM with explicit parallel nodes.

3.13 Testing contracts

Pure model tests, deterministic directive loops, and recorded provider traffic test different boundaries.

Layer	Target	Evidence
Unit	Machine, merge, node construction, AgentTool, Error	Direct input/output assertions
Strategy	Dispatch, concurrency, approval, replay	LLMStub or manually driven directives
Integration	Nesting, branching, result adaptation	Deterministic stub or cassette
End-to-end	Provider round trips and multi-turn composition	Recorded HTTP cassette
Recording	Capture real provider response shape	Explicit network run with secret filtering

- LLMStub direct mode uses setup plus execute for a manually driven directive loop. Plug mode uses setup_req_stub and Req.Test so the real LLMAction and ReqLLM path still executes. Stub outcomes are tool_calls, final_answer, and error.
- ReqCassette record mode records when absent and replays when present; replay mode refuses missing cassettes; bypass always reaches the network. Never hand-author a cassette presented as recorded traffic.
- req_options carries the test plug through the strategy and LLMAction into req_http_options. Production logic does not inspect test mode.
- Central cassette filters remove authorization, x-api-key, cookie, set-cookie, provider key patterns, JSON api_key values, and Bearer tokens. Filters reduce exposure but do not prove arbitrary bodies are safe to publish.
- Test linear, branching, error, and nested workflows; single and multiple tool turns; agent tools; orchestrator-to-workflow composition; model errors; approval, suspension, checkpoint replay, and span cleanup.
- The source's development sequence is test-first: observe a failing contract test, implement, refactor, then exercise integration and the repository quality gate. This migration executes the document gate, not these implementation tests.

3.14 End-to-end walkthrough

The ETL example is the canonical executable-shape illustration. The remaining source scenarios show how the same boundary composes at other scales.

```
use Jido.Composer.Workflow,
  name: "etl_pipeline",
  description: "Extract, transform, validate, and load data",
  nodes: %{extract: ExtractFromAPI, transform: NormalizeSchema,
    validate: ValidateRecords, load: WriteToStore,
    quarantine: QuarantineBadRecords},
  transitions: %{{:extract, :ok} => :transform,
    {:transform, :ok} => :validate, {:validate, :ok} => :load,
```

```
    {:validate, :invalid} => :quarantine, {:load, :ok} => :done,
    {:quarantine, :ok} => :done, {:_, :error} => :failed},
  initial: :extract

# Put the declaration above inside defmodule ETLPipeline.
agent = ETLPipeline.new()
ETLPipeline.run_sync(agent, %{source: "records"})
# Representative success, assuming the supplied actions return these maps:
# {:ok, %{source: "records", extract: %{records: [1, 2]},
#   transform: %{records: [1, 2]}, validate: %{valid: true},
#   load: %{count: 2}}}
```

- This is a configuration example, not a runnable application: the named action modules are supplied by the consumer. The original names are preserved as quoted example identifiers.
- Each successful node result accumulates in its state scope. validate's invalid outcome selects quarantine; error uses the wildcard failed edge.
- An orchestrator may expose this workflow as a tool using AgentNode. The declared description tells the model when to choose it; the parent does not inspect the ETL transition table.

Source scenario	Composition	What it demonstrates
ETL pipeline	Workflow with actions	Fixed sequence and validation branch
Research coordinator	Orchestrator with search and query tools	Runtime operation selection
Project manager	Orchestrator over onboarding, deploy, and report workflows	Adaptive selection of fixed pipelines
Content publisher	Workflow with editorial orchestrator	One adaptive step inside a fixed process
Customer support	Orchestrator to workflow to diagnostic orchestrator	Three-level nesting
Due diligence	Workflow with financial, legal, and background branches	Named parallel results before a final decision
Product development	Coordinator over specialist orchestrators	Stable specialist boundaries
Dynamic assembly	Orchestrator with DynamicAgentNode and Skills	Runtime capability selection

- The editorial example requires a concrete mapping from structured verdict to workflow outcome; a prompt alone does not establish that adapter. Preserve it as an illustrative scenario until that mapping is specified.
- The due-diligence example uses named branch pairs in the canonical FanOut contract; its older unnamed branch-list sketch is not the constructor reference.
- For dynamic assembly, put model and base_prompt inside assembly_opts. The example's registry must actually contain every skill name the parent can select.
- The original detailed examples remain available in use-cases.md during review; their behavioral gaps are not silently presented as working code.

Glossary

Every term the layer declares, with the context that owns it.

ActionNode

owned by execution

A node adapter for one Jido Action with per-use options.

AgentNode

owned by execution

A node adapter for one Jido Agent with synchronous, asynchronous, or streaming execution configuration.

AgentTool

owned by execution

The adapter between node metadata and model tool descriptions, arguments, and correlated results.

Ambient context

owned by core-model

Read-only values inherited by a composition and its descendants.

Approval gate

owned by core-model

The policy and pending-call state that withhold selected tool invocations until human approval. *Avoid*: HumanNode — a separate human-input participant rather than enforcement around another invocation.

ApprovalRequest

owned by core-model

A serializable request for a human decision, with permitted outcomes and the context visible to the respondent.

ApprovalResponse

owned by core-model

A response to one approval request containing a decision, optional data, respondent information, and response time.

CheckpointAndStop

owned by execution

A directive requesting durable checkpoint storage, parent notification, and process termination.

ChildRef

owned by core-model

A serializable reference to a child agent's identity, checkpoint, execution status, and communication phase.

Context

owned by core-model

The data available to a composition, partitioned into ambient values, accumulated working results, and boundary transformations.

Directive

owned by execution

A description of an external operation emitted for a runtime to execute.

DynamicAgentNode

owned by execution

A node that selects skills from a registry, assembles an orchestrator, and executes the requested task.

FanOutBranch

owned by execution

A directive describing one child-node execution and its prepared input within a parallel or traverse operation.

FanOutNode

owned by core-model

A node composing a fixed collection of named child nodes into one merged result.

Fork function

owned by core-model

A named module-function-arguments transformation that derives a child's ambient context at an agent boundary.

HumanNode

owned by execution

A node that requests human input and immediately yields a suspension outcome.

LLMAction

owned by execution

The Jido Action boundary between an orchestrator and ReqLLM generation and conversation handling.

Machine

owned by core-model

The workflow value containing configuration, current state, context, and transition history.

MapNode

owned by core-model

A node applying one configured child node to a runtime list and collecting results in input order.

Node

owned by core-model

A configured participant with a context-in, result-out interface and an optional transition outcome.

NodeIO

owned by core-model

An output envelope containing a map, text, or structured object with its type and optional schema.

Obs

owned by execution

A strategy-specific value tracking observation spans and measurements for one execution.

Orchestrator

owned by core-model

A composition whose language model selects calls to available nodes until it returns an answer or reaches a limit.

OtelCtx

owned by execution

The boundary for attaching and restoring process-local OpenTelemetry context.

Outcome

owned by core-model

An atom associated with a node result that determines a workflow transition. The reserved suspend outcome holds the current state instead.

Replay directives

owned by execution

Operations reconstructed from checkpointed execution state to re-establish interrupted work.

Req options

owned by execution

Opaque HTTP configuration forwarded through an orchestrator to ReqLLM's Req integration.

Resume

owned by execution

The addressing operation that delivers a continuation to a live agent or restores a checkpointed agent before delivery.

Skill

owned by core-model

A named capability bundle containing a description, prompt fragment, and compatible tool modules.

Skill assembly

owned by core-model

The transformation of selected skills and options into a configured orchestrator, independently of execution.

Snapshot

owned by core-model

A strategy-independent view of execution status, completion, result, and diagnostic details.

Strategy

owned by execution

The command-processing behavior that updates an agent value and describes work for its runtime.

Strategy state

owned by execution

The strategy-owned value stored under the agent state's reserved `__strategy__` key.

Suspend directive

owned by execution

A directive carrying suspension metadata, optional notification configuration, and hibernate intent. *Avoid*: suspend outcome — the outcome is a result marker, not a runtime operation.

SuspendForHuman

owned by execution

A compatibility constructor for a suspension directive whose reason is human input.

Suspension

owned by core-model

The serializable record of a paused computation and its resume correlation, reason, and timeout policy.

Terminal state

owned by core-model

A declared workflow state that ends execution without running a node. Success states are the subset denoting successful completion.

Tool

owned by core-model

The language-model-facing description of a callable node, including its name, description, and parameter schema.

Tool call

owned by core-model

A model-selected invocation identified by a call ID, tool name, and argument map.

Traverse

owned by core-model

The composition operation that applies the same node to each element of a collection. *Avoid*: parallel — parallel composition selects a fixed set of different branches.

Workflow

owned by core-model

A composition whose declared states and outcome-driven transitions determine execution order.

Working context

owned by core-model

The portion of context containing input data and results accumulated under node-specific scope keys.